LLM energy management agent

System flow, data contracts and execution plan. Combines the ML forecasting pipeline (ML 1 demand, ML 2 production) with a small-LLM decision agent driven by the break-even prompt, user goals, and a deterministic guard layer.

1. Overview

Every 15 minutes the system makes one decision: how to use the battery for the next interval. The responsibilities are split so that each part does what it is good at:

- **ML models** predict what will happen: demand (ML 1) and solar + wind production with a forecast range (ML 2).
- **Deterministic Python** does all arithmetic (break-even price, limits, matching fraction, goal progress), enforces safety, and settles the money.
- **The small LLM** compares the prepared numbers, weighs timing, uncertainty and user goals, and chooses one action with a short explanation. It never calculates and never touches the hardware directly.
- **The user** sets goals in plain language (“maximise profit”, “store 6 MWh by 18:00”). Goals are parsed once into structured settings that flow into the feature builder, the prompt, the guard and the settlement.

Design rule: the LLM proposes, the code disposes. Anything the LLM outputs passes a validator and a guard layer before it reaches the battery, and a rule-based fallback keeps the site running if the LLM fails or times out.

2. Full-flow diagram

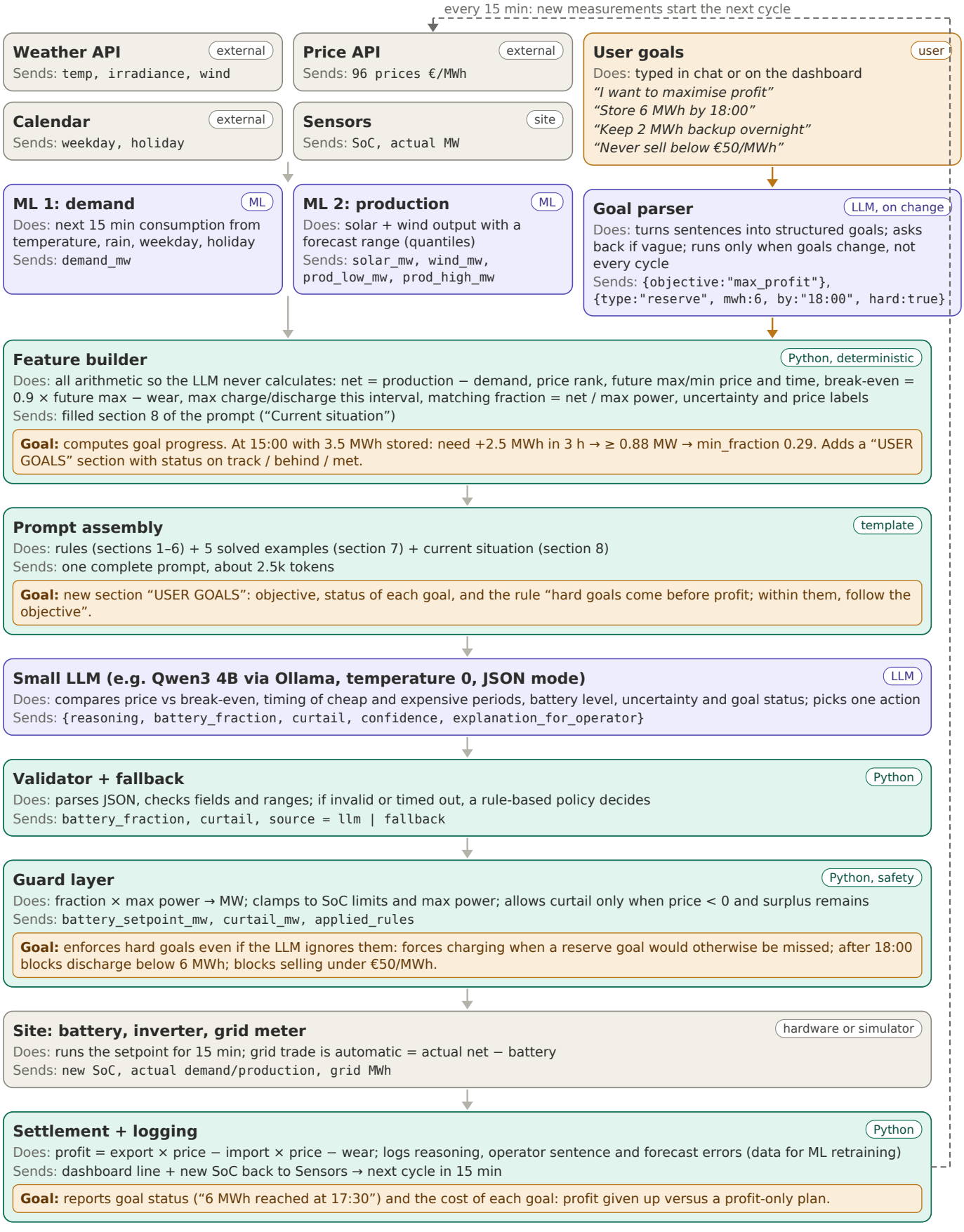


Figure 1. One control cycle. Left: data and forecasts. Right: user goals, parsed only when they change. Amber boxes show where goals act in each stage.

3. What each part does and sends

Component	Runs	Does	Receives	Sends
Weather API	every cycle	Weather forecast for the site	location	temperature, irradiance, wind speed, precipitation
Price API	once a day + check	Day-ahead market prices for all 96 intervals	date	96 prices €/MWh
Calendar	once a day	Day context	date	weekday, holiday flag
Sensors	every cycle	Live site measurements	hardware	SoC (MWh), actual demand and production (MW)
ML 1: demand	every cycle	Predicts consumption for the next 15 min	weather, calendar, recent load	demand_mw
ML 2: production	every cycle	Predicts solar and wind output with a range	weather, installed panels/turbines	solar_mw, wind_mw, prod_low_mw, prod_high_mw
User goals	when the user wants	Natural-language goals	user	goal sentences
Goal parser (LLM)	on goal change	Turns sentences into structured goals; asks back if vague	goal sentences	goal config JSON (objective + list of goals)
Feature builder	every cycle	All arithmetic and labels; goal progress	forecasts, prices, SoC, goal config	values for section 8 + USER GOALS section, min_fraction per goal
Prompt assembly	every cycle	Joins fixed rules, examples and the current situation	feature values	complete prompt text
Small LLM	every cycle	Chooses the battery action	prompt	JSON: reasoning, battery_fraction, curtail, confidence, explanation_for_operator
Validator + fallback	every cycle	Checks the JSON; rule-based decision if invalid	LLM output	battery_fraction, curtail, source
Guard layer	every cycle	Converts to MW, enforces SoC/power limits and hard goals	decision, limits, goal config	battery_setpoint_mw, curtail_mw, applied_rules
Site / simulator	15 min	Executes the setpoint; grid trade is automatic	setpoint	new SoC, actual values, grid import/export MWh
Settlement + logging	every cycle	Profit, wear, goal status, logs for retraining	site results, prices, decision	dashboard line, metrics, data back to Sensors

4. User goals

Goals are entered in plain language and parsed once into a structured goal config. Each goal is either **soft** (it shapes the LLM's choice through the prompt) or **hard** (the guard layer enforces it regardless of what the LLM says).

User says	Parsed as	Hard / soft	How it is applied
“I want to maximise profit”	`{objective:"max_profit"}`	soft	Default objective in the prompt; break-even logic decides.
“Use my own energy first”	`{objective:"self_consumption"}`	soft	Prompt tells the LLM to prefer the matching fraction and avoid grid trading.
“Store 6 MWh by 18:00”	`{type:"reserve", mwh:6, by:"18:00"}`	hard	Feature builder computes progress; guard forces charging when the goal would otherwise be missed.
“Keep 2 MWh backup overnight”	`{type:"reserve", mwh:2, by:"22:00", until:"06:00"}`	hard	Guard blocks discharge below 2 MWh inside the window.
“Never sell below €50/MWh”	`{type:"min_sell_price", eur:50}`	hard	Guard blocks discharge to the grid when price < €50; surplus is stored or curtailed.

Worked example: “store 6 MWh by 18:00”

At 15:00 the battery holds 3.5 MWh (battery: 10 MWh, 3 MW max, round-trip efficiency 0.90, so $\eta_{charge} = \sqrt{0.90} \approx 0.949$).

- Energy still needed: 6 − 3.5 = **2.5 MWh**, time left: **3 h**.
- Even pace (shown to the LLM as information): 2.5 / 3 = 0.833 MWh/h stored, so 0.833 / 0.949 = **0.88 MW** charging power, which is 0.88 / 3 = **min_fraction 0.29**.
- Latest start (what the guard enforces): at full power the 2.5 MWh take 2.5 / (0.949 × 3) = 0.88 h ≈ 53 min. With a 30 min safety buffer the guard must force full charging from 18:00 − 53 min − 30 min ≈ **16:37**, i.e. from the 16:30 interval if the goal is still open.
- Why two numbers: the even pace tells the LLM how urgent the goal is, while the latest-start rule leaves it free to wait for cheaper energy until waiting is no longer safe.

The LLM sees the goal in a new prompt section, placed just before “Respond ONLY with the JSON decision”:

```
USER GOALS (hard goals come before profit; within them, follow the objective)
- Objective: maximise profit
- [HARD] Store 6 MWh by 18:00 -> status: ON TRACK. Have 3.5 MWh, need +2.5 MWh in 3.0 h.
         Even pace: battery_fraction >= 0.29. Guard forces full charging from 16:30 if still open.
- [HARD] Never sell below EUR 50/MWh -> current price EUR 95: allowed.
```

5. Data contracts (example values for 15:00)

Fixed JSON shapes between components make each part testable on its own.

Goal parser → feature builder, guard, settlement

```
{
  "objective": "max_profit",
  "goals": [
    {"id": "g1", "type": "reserve", "mwh": 6.0, "by": "18:00", "until": "18:00", "hard": true,
     "source_text": "Store 6 MWh by 18:00"},
    {"id": "g2", "type": "min_sell_price", "eur_per_mwh": 50, "hard": true,
     "source_text": "Never sell below 50 EUR/MWh"}
  ]
}
```

Feature builder → prompt assembly (subset)

```
{"time": "15:00", "demand_mw": 4.6, "production_mw": 5.0, "net_mw": 0.4, "net_label": "SURPLUS",
 "price_now": 95, "price_rank": 70, "price_avg": 80, "future_max_price": 170, "future_max_time": "20:30",
 "break_even": 148.0, "price_vs_breakeven": "EUR 53 BELOW",
 "stored_mwh": 3.5, "max_charge_mw": 3.0, "max_discharge_mw": 3.0, "matching_fraction": 0.13,
 "goals": [{"id": "g1", "status": "on_track", "need_mwh": 2.5, "hours_left": 3.0,
            "even_pace_fraction": 0.29, "force_from": "16:30"}]}
```

Small LLM → validator

```
{"reasoning": "The price is EUR 53 below break-even, so stored energy is worth more tonight. The 6 MWh goal
   needs 0.29 at an even pace; the surplus alone (0.13) is not enough, so I charge at the goal pace.",
 "battery_fraction": 0.29, "curtail": false, "confidence": "medium",
 "explanation_for_operator": "Charging 0.87 MW to reach the 6 MWh goal by 18:00."}
```

Guard → site, and settlement log

```
{"battery_setpoint_mw": 0.87, "curtail_mw": 0.0, "clipped": false,
 "applied_rules": ["soc_limits_ok", "goal_g1_ok"], "source": "llm"}

{"time": "15:00", "soc_after_mwh": 3.71, "grid_import_mwh": 0.1175, "grid_export_mwh": 0.0,
 "energy_cost_eur": -11.16, "wear_eur": -0.54, "profit_eur": -11.71, "goal_g1": "on_track"}
```

Check of the settlement numbers: battery 0.87 MW, surplus 0.4 MW → import 0.47 MW × 0.25 h = 0.1175 MWh × €95 = €11.16. Wear €2.5 per MWh of throughput × 0.87 × 0.25 = €0.54. SoC after: 3.5 + 0.949 × 0.87 × 0.25 = 3.71 MWh.

6. Fixes needed in the current prompt

I checked the five solved examples against each other. The break-even formula is consistent in all of them: break-even = 0.9 × highest later price − €5 (160 → 139, 120 → 103, 140 → 121, 150 → 130, 170 → 148), and all matching fractions are correct (2.6/3 = 0.87, −1.8/3 = −0.60, 6.3/2 = 3.15, −0.5/1 = −0.50, 0.4/3 = 0.13). Four things should still change:

Issue	Why it matters	Fix
Max charge/discharge in examples 2 and 4 use a 1-hour basis	Example 3 uses 15 min: 0.5 MWh free → 0.5 / 0.25 h = 2 MW. Example 2 has 1.5 MWh free → should be min(3, 1.5 / 0.25) = 3 MW, not 1.5. Example 4 lists 1 MW with 2 MWh stored; with a 1 MWh minimum that is min(3, 1 / 0.25) = 3 MW, and 1 MW only fits a 1-hour basis. Inconsistent examples teach the model the wrong scale.	Define in code: max_charge = min(P_max, free / (η_c × 0.25 h)), max_discharge = min(P_max, (E − E_min) × η_d / 0.25 h). Regenerate the examples with it.
Break-even uses one future price	One interval at 3 MW sells only 0.75 MWh; emptying a full 10 MWh battery takes about 13 intervals. Valuing every stored MWh at the single highest price overstates its worth when the battery is fairly full.	Use the k-th highest later price, with k = intervals needed to discharge the stored energy.
Break-even ignores refill opportunities	Morning €120, midday −€8, evening €165: break-even = 0.9 × 165 − 5 = 143.5 > 120, so the agent holds. Selling at €120 and refilling at −€8 earns more.	Compute the value of stored energy only up to the next cheap period, or pass a second “break-even before next refill” number.
Energy balance line omits curtailment	“production − demand = battery_charging + grid_export” is wrong whenever curtail = true (example 3).	Write: production − demand = battery_charging + grid_export + curtailed.

7. Execution plan

Suggested 10-week schedule; shift the weeks to your deadline. Each phase ends with something that runs and can be tested on its own.

Phase	Week	Tasks	Deliverable	Done when
0. Setup and data	1	Repo structure; download historical weather (e.g. Open-Meteo), day-ahead prices (e.g. ENTSO-E Transparency or OPCOM), a load dataset and PV/wind output; align everything to 15-min intervals	data/ folder + loading scripts	one table with all signals per 15 min, no gaps
1. ML 1 and ML 2	2–3	Features (hour, weekday, holiday, temperature, irradiance, wind, lagged values); baseline = persistence; gradient boosting; quantile models for prod_low/high	two trained models + predict() API	MAE beats persistence; 80% range covers ~80% of actuals
2. Simulator and benchmarks	3–4	Battery model (SoC, η, limits, wear); settlement; baselines: no battery, greedy matching fraction; LP with perfect foresight as the upper bound	simulate(policy, day) → profit, SoC trace	energy balances every step; LP ≥ every other policy
3. Feature builder and prompt	5	Implement every section-8 field; fix the 4 prompt issues; regenerate the solved examples from code; unit tests per field	build_features(), render_prompt()	the 5 examples reproduce exactly from code
4. LLM integration	6	Ollama + Qwen3 4B (compare with another small model); JSON mode, temperature 0, num_ctx 8192; validator, timeout and fallback	decide(prompt) → validated decision	≥ 99% valid JSON over 1 simulated week; fallback tested
5. Goals and guard	7	Goal parser prompt and schema; goal progress in feature builder; USER GOALS prompt section; hard-goal rules in guard	parse_goals(), guard()	hard goals never violated in simulation
6. Evaluation	8–9	30+ simulated days (sunny, cloudy, negative-price, holiday); compare policies; ablations: no examples, no break-even, no uncertainty; goal-cost analysis	results tables and plots	metrics below filled in for every policy
7. Dashboard and report	10	Live view: prices, SoC, decisions, operator sentences, goal status; write-up	demo + report	end-to-end run on one day without manual steps

Evaluation metrics

Metric	Definition	Target / use
Profit captured	agent profit / LP perfect-foresight profit (both relative to no battery)	main result; compare with greedy
Goal satisfaction	share of hard goals met; cost of each goal vs profit-only run	100% for hard goals
LLM reliability	valid JSON rate, fallback rate, guard clip rate	≥ 99% valid; low clip rate
Latency	seconds per decision on the target machine	well below 15 min (expect seconds)
Forecast quality	MAE / range coverage for ML 1 and ML 2, and profit lost due to forecast error	explains gaps to LP
Explanation quality	manual check of 50 random operator sentences: correct and consistent with action	qualitative

Risks and mitigations

- **The LLM picks a poor action.** Guard + fallback keep it safe; evaluation measures the profit gap to the LP; the solved examples are the main lever for improving behaviour.
- **Prompt too long for the context window.** The prompt is about 2.5k tokens. Ollama's default context can be as small as 4k tokens, so set num_ctx to 8192 explicitly.
- **Forecast errors.** ML 2's range feeds the uncertainty label (moderate actions when wide); settlement always uses actual values, so errors show up in the metrics instead of hiding.
- **Non-repeatable runs.** Temperature 0, fixed model version, and every prompt and response logged.

Next step after this plan: the full-day simulation (phase 2 simulator running the phase 3–5 pipeline), which produces the first numbers for the evaluation table.